

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

ICT Class 10 - Formative Assessment 4

Formative Assessment (FA-4) - Answer Key

Class: Class 10

Assessment Type: Formative Assessment (FA-4)

Total Marks: 30

Duration: 90 minutes

Topics Covered: SQL Database Management

ICT Teaching Resources Portal

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: MCQ (5 x 1 = 5 marks)

- Answer:** c - SELECT
 - Explanation:** The SELECT command retrieves data from database tables. GET, RETRIEVE, and FETCH are not SQL commands.
- Answer:** b - SELECT, UPDATE, DELETE
 - Explanation:** The WHERE clause is used with DML commands to filter rows. It is not used with INSERT or CREATE.
- Answer:** b - SUM()
 - Explanation:** SUM() returns the total sum of a numeric column. COUNT() counts rows, TOTAL() and ADD() are not aggregate functions.
- Answer:** c - DROP TABLE
 - Explanation:** DROP TABLE completely removes a table and its structure. DELETE removes rows, TRUNCATE removes all rows but keeps structure.
- Answer:** c - Group rows with same values for aggregate functions
 - Explanation:** GROUP BY groups rows sharing the same values in specified columns for use with aggregate functions.

Section II: Short Answer (5 x 2 = 10 marks)

1. SQL Queries for Students table

- Answer:**

```
-- Students of class 10 with marks above 80
SELECT * FROM Students WHERE class = '10' AND marks > 80;

-- Count students in each section
SELECT section, COUNT(*) FROM Students GROUP BY section;

-- Maximum and minimum marks
SELECT MAX(marks), MIN(marks) FROM Students;

-- Alphabetical order of names
SELECT * FROM Students ORDER BY name ASC;
```

- Marking:** 0.5 marks for each correct query.

2. WHERE vs HAVING clause

- Answer:** WHERE filters rows before grouping (used with SELECT, UPDATE, DELETE). HAVING filters groups after GROUP BY (used with aggregate functions). Example: WHERE marks > 80 filters individual rows; HAVING COUNT(*) > 5 filters groups.
- Marking:** 1 mark for difference, 1 mark for examples.

3. Primary key

- **Answer:** A primary key uniquely identifies each row in a table. It cannot contain NULL values and must be unique. A table can have only one primary key (though it can be composite - multiple columns). Example: roll_no in Students table.
- **Marking:** 1 mark for definition, 1 mark for rules.

4. SQL Output

- **Answer:**

```
SELECT UPPER('hello world');      -- 'HELLO WORLD'
SELECT LENGTH('Karnataka');      -- 9
SELECT CONCAT('ICT', ' Class', ' 10'); -- 'ICT Class 10'
```

- **Marking:** 0.5 marks for each correct output.

5. ACID properties

- **Answer:** Atomicity - transaction is all-or-nothing (bank transfer debits and credits). Consistency - database remains valid (constraints always satisfied). Isolation - concurrent transactions don't interfere (two users buying same item). Durability - committed data persists (survives system crash).
- **Marking:** 0.5 marks for each property with example.

Section III: Long Answer (5 x 1 = 5 marks)

1. Library Management System Queries

- **Answer:**

```
-- (a) Books with price above 500 sorted by title
SELECT * FROM Books WHERE price > 500 ORDER BY title;

-- (b) Members who have not returned any book
SELECT m.name FROM Members m
WHERE m.member_id NOT IN (SELECT member_id FROM Issue WHERE return_date IS NULL);

-- (c) Count how many times each book has been issued
SELECT b.title, COUNT(i.issue_id) AS times_issued
FROM Books b LEFT JOIN Issue i ON b.book_id = i.book_id
GROUP BY b.book_id, b.title;

-- (d) Total revenue from all books
SELECT SUM(b.price * i.quantity) AS total_revenue
FROM Books b JOIN Issue i ON b.book_id = i.book_id;

-- (e) Members who joined in last 6 months
SELECT * FROM Members
WHERE membership_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

- **Marking:** 1 mark for each correct query.

Part B: Hands-on Practical Lab Task (5 marks)

SQLite Student Management System

```
import sqlite3

try:
    conn = sqlite3.connect('school.db')
    c = conn.cursor()
```

```

# Create table
c.execute('''CREATE TABLE IF NOT EXISTS Students
            (roll INTEGER PRIMARY KEY, name TEXT, class TEXT, marks INTEGER)''')

# Insert records
students = [(1, 'Rahul', '10A', 85), (2, 'Priya', '10A', 92),
            (3, 'Amit', '10B', 78), (4, 'Sneha', '10B', 88),
            (5, 'Vijay', '10A', 71), (6, 'Neha', '10A', 95),
            (7, 'Ravi', '10B', 82), (8, 'Pooja', '10B', 76)]
c.executemany('INSERT OR IGNORE INTO Students VALUES (?, ?, ?, ?)', students)

# CRUD Operations
# Create
c.execute('INSERT INTO Students VALUES (9, "Kiran", "10A", 89)')

# Read
c.execute('SELECT * FROM Students')
print("All Students:", c.fetchall())

# Update
c.execute('UPDATE Students SET marks = 90 WHERE roll = 3')

# Delete
c.execute('DELETE FROM Students WHERE roll = 9')

# Aggregate Report
c.execute('SELECT COUNT(*), AVG(marks), MAX(marks), MIN(marks) FROM Students')
print("Stats:", c.fetchone())

conn.commit()

except sqlite3.Error as e:
    print(f"Database error: {e}")
finally:
    conn.close()

```

- **Marking:** 1 mark for database setup, 1 mark for CRUD operations, 1 mark for aggregates, 1 mark for exception handling, 1 mark for proper closing.

Part C: Notebook Maintenance (5 marks)

1. Neatness and organisation of notebook (2 marks)
2. Completion of all assigned classwork and homework (2 marks)
3. Proper headings, dates, and indexing (1 mark)